

Архиватор на базе алгоритма Хаффмана

Разгуляева Ада Игоревна

Группа: 25.Б42-мм

Кафедра: системного программирования

Научный руководитель: Литвинов Юрий Викторович

Номер семестра практики: 2

1. Введение

Данная работа выполнена в рамках учебной практики по курсу «Технологии программирования». Алгоритм Хаффмана — классический метод сжатия данных без потерь, основанный на построении префиксных кодов переменной длины [1]. Код каждого символа строится так, что ни один код не является префиксом другого, что позволяет однозначно восстанавливать исходные данные. Реализация требует работы с битовыми операциями, динамическими структурами данных и бинарными файлами, что делает её полезной для освоения низкоуровневых возможностей языка C.

2. Постановка задачи

Целью работы является разработка консольного приложения на C, выполняющего сжатие и разжатие файлов по алгоритму Хаффмана и экспериментальная оценка его эффективности.

Для достижения цели требуется решить следующие задачи:

1. Выполнить обзор существующих реализаций алгоритма Хаффмана.
2. Спроектировать архитектуру консольного приложения.
3. Реализовать консольное приложение.
4. Выполнить тестирование и экспериментальное исследование.

3. Обзор

В качестве ориентиров рассмотрены открытые реализации на языке C:

- SmnTin/huffman-archiver [2]: архиватор с каноническими кодами Хаффмана, оптимизированный для больших файлов.
- MichaelDipperstein/huffman [3]: простая и читаемая реализация, включающая как классические, так и канонические коды Хаффмана.
- ConnorDye/C-Zip [4]: учебный проект с модульной архитектурой.

Вывод из обзора. Анализ существующих реализаций [2–4] позволил выделить общие архитектурные решения. Из C-Zip [4] заимствована модульная структура: битовый I/O, очередь с приоритетами и дерево Хаффмана разделены по функциям внутри модуля. Подход к буферизованной побитовой записи и сериализации таблицы кодов в заголовок, встречающийся в ряде рассмотренных реализаций, обобщён и применён в данной работе. Из huffman-archiver SmnTin [2] — идея буферизации вывода для ускорения работы с битами.

4. Описание предлагаемого решения

4.1. Архитектура

Проект состоит из:

- huffman.h / huffman.c — публичный интерфейс и реализация (дерево, куча, битовый I/O, сжатие, распаковка);
- main.c — интерфейс командной строки.

4.2. Технологии

- Язык C99, компилятор GCC ≥ 9 (также собирается Clang и MSVC).
- Система сборки: CMake ≥ 3.28 .
- Статические анализаторы: clang-format (WebKit), clang-tidy.
- CI через GitHub Actions: сборка, проверка форматирования, линтинг и тесты на Linux и Windows.

4.3. Технические детали

- Используется разрежённый заголовок (HdrSparse): записываются только ненулевые частоты. Это снижает оверхед с 2 КБ до десятков–сотен байт для типичных файлов.
- Для построения дерева используется очередь с приоритетами с операциями за $O(\log n)$ вместо $O(n)$.
- Битовый поток упаковывается в байты с буферизацией, неполный байт сбрасывается при завершении.
- Отдельно обрабатывается случай одного уникального символа.
- Используется формат чисел little-endian, для обеспечения единого формата файла независимо от архитектуры.

5. Эксперименты

Для оценки эффективности алгоритма и анализа работы программы были поставлены эксперименты, демонстрирующие степень сжатия, время сжатия и время разжатия в зависимости от размера и типа входных данных. Было протестировано шесть категорий данных: сильно повторяющиеся (repeated, .bin), текстовые (text, .txt), исходный код (code, .c), случайные байты (random, .bin), уже сжатые архивы (archive, .zip) и изображения в формате JPEG (photo, .jpg). Размеры варьировались от 1 КБ до 5 МБ. Для каждого файла выполнялось 10 прогонов с расчётом математического ожидания (μ) и среднеквадратичного отклонения (σ) времени выполнения.

- Эксперименты проводились на системе со следующими характеристиками:
- Компилятор: gcc версии 13.3.0
 - Флаги компиляции: -O3 -DNDEBUG (Release)
 - Процессор: 13th Gen Intel Core i5-1335U, тактовая частота 0,4–4,6 ГГц
 - Оперативная память: 15 ГиБ
 - Операционная система: Ubuntu 24.04.4 LTS, ядро 7.0.0-31-generic

Таблица 1: Степень сжатия по типам данных и размерам, % от исходного размера

Тип	1 КБ	10 КБ	100 КБ	1 МБ	5 МБ
repeated	85.64	87.31	87.48	87.50	87.50
text	17.09	40.96	43.35	43.59	43.61
code	−32.23	24.91	33.92	34.99	35.10
random	−219.63	−22.60	−2.26	−0.22	−0.04
archive	—	−22.26	−2.26	−0.22	—
photo	—	−2.44	−1.02	+0.31	—

Таблица 2: Время сжатия, мс ($\mu \pm \sigma$, 10 прогонов)

Тип	1 КБ	10 КБ	100 КБ	1 МБ	5 МБ
repeated	1.35 ± 0.29	1.32 ± 0.16	2.24 ± 0.38	7.09 ± 2.31	22.65 ± 3.60
text	1.68 ± 0.18	1.69 ± 0.46	4.60 ± 0.79	23.73 ± 2.98	96.58 ± 5.52
code	0.97 ± 0.18	1.58 ± 0.10	9.19 ± 2.39	49.95 ± 7.30	222.36 ± 7.27
random	2.09 ± 0.21	2.54 ± 0.47	11.63 ± 1.79	69.88 ± 3.98	334.83 ± 6.55
archive	—	2.34 ± 0.23	11.95 ± 0.78	69.13 ± 3.89	—
photo	—	2.07 ± 0.19	13.24 ± 3.12	75.73 ± 6.68	—

Таблица 3: Время разжатия, мс ($\mu \pm \sigma$, 10 прогонов)

Тип	1 КБ	10 КБ	100 КБ	1 МБ	5 МБ
repeated	1.31 ± 0.26	1.30 ± 0.21	2.02 ± 0.47	6.49 ± 1.67	25.17 ± 4.74
text	1.67 ± 0.17	1.49 ± 0.37	2.96 ± 0.38	14.04 ± 3.01	51.86 ± 5.49
code	0.89 ± 0.18	1.34 ± 0.12	5.80 ± 2.09	31.86 ± 6.27	127.30 ± 3.21
random	1.92 ± 0.15	2.11 ± 0.53	7.82 ± 1.28	50.01 ± 2.65	229.71 ± 4.80
archive	—	2.05 ± 0.21	8.46 ± 0.79	50.18 ± 3.55	—
photo	—	1.73 ± 0.15	9.52 ± 2.18	56.17 ± 7.06	—

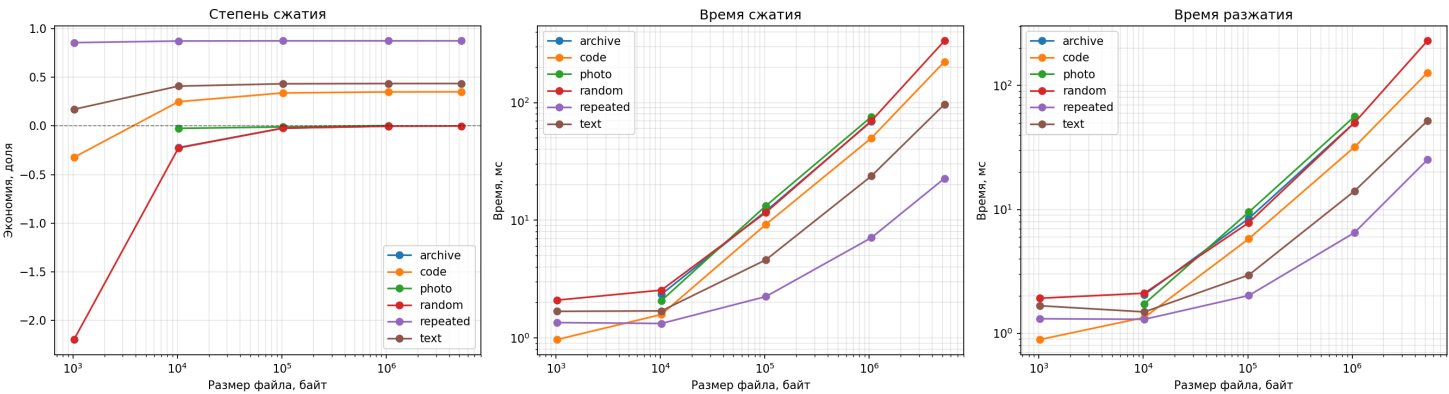


Рис. 1: Зависимость степени сжатия и времени работы от размера файла

5.1. Выводы из результатов эксперимента

- Эффективность сжатия определяется энтропией входных данных. Наибольшее сжатие достигается на однородных данных (repeated, 85.6–87.5%): при одном уникальном символе код каждого байта занимает ровно один бит, что даёт восьмикратное уменьшение размера. Текстовые данные сжимаются на 41–44%, исходный код — на 25–35%. Процент сжатия слабо зависит от размера файла, начиная с 10 КБ.
- На уже сжатых (JPEG, ZIP) и случайных данных алгоритм неэффективен: размер файла, как правило, увеличивается на величину заголовка, зависящую от числа уникальных байтов.
- Маленькие файлы (менее 2–3 КБ) сжимаются неэффективно или увеличиваются в размере. В используемом формате заголовок содержит таблицу частот переменного размера: 8 байт на исходный размер, 2 байта на количество уникальных символов и по 9 байт на каждый уникальный символ (байт и восьмибайтовая частота). При полном алфавите из 256 символов заголовок занимает 2314 байт, что превышает объём данных в 1 КБ.
- Время работы асимптотически линейно зависит от размера входного файла ($O(n)$): и сжатие, и разжатие выполняются за один проход по данным. Измеренный наклон на логарифмических графиках меньше 1 из-за фиксированных накладных расходов на малых размерах.
- Разжатие выполняется быстрее сжатия примерно в 1,2–1,9 раза на текстовых и случайных данных, на однородных данных времена сопоставимы. При сжатии дополнительно тратится время на построение дерева Хаффмана и генерацию кодов, тогда как при разжатии достаточно одного обхода дерева по битовому потоку.
- Случайные и уже сжатые данные обрабатываются медленнее, чем код и намного медленнее, чем текст: из-за 256 уникальных символов средняя длина кода приближается к 8 битам, поэтому на каждый байт входа приходится записывать (или считывать) почти столько же бит, сколько занимал сам байт и экономии почти нет, а побитовых операций выполняется больше на тот же объём входных данных. Дополнительный, но менее значимый вклад вносит и построение дерева: с 256 символами требуется до 255 слияний в куче против существенно меньшего числа для текста и кода, где алфавит компактнее.

6. Заключение

В ходе выполнения работы получены следующие результаты:

- изучена теория префиксных кодов и алгоритма Хаффмана;
- реализовано консольное приложение на C;
- настроены CI и статические анализаторы для Linux и Windows;
- написаны тесты, проверяющие работу архиватора;
- поставлены эксперименты, показавшие границы эффективной применимости алгоритма.

Репозиторий/ссылка на пуллреквест: https://github.com/ada1ra/archiver_huffman/.

Техническая документация: https://github.com/ada1ra/archiver_huffman/blob/main/README.md.

Список литературы

1. Huffman D. A. A Method for the Construction of Minimum-Redundancy Codes // Proceedings of the IRE. — 1952. — Vol. 40, No. 9. — P. 1098–1101.
2. SmnTin. huffman-archiver [Электронный ресурс]. — URL: <https://github.com/SmnTin/huffman-archiver> (дата обращения: 21.09.2026).
3. MichaelDipperstein. huffman: Huffman code discussion and implementation [Электронный ресурс]. — URL: <https://github.com/MichaelDipperstein/huffman> (дата обращения: 21.09.2026).
4. Connor Dye. C-Zip: File Compression and Decompression Application [Электронный ресурс]. — URL: <https://github.com/ConnorDye/C-Zip-File-Compression-and-Decompression-Application> (дата обращения: 21.09.2026).